

Terraform Interview Q&A

30 Real-Time Terraform Interview Questions & Answers

For MNC Interviews (Accenture, TCS, Infosys, Capgemini, Deloitte, Cognizant, IBM, EPAM, HCL, Wells Fargo, JPMC, Verizon, ADP)

Level: Intermediate to Advanced | **Focus:** Production Scenarios, AWS, CI/CD

Question 1

Interview Question:

Walk me through Terraform's architecture. How does it actually talk to AWS under the hood?

Answer:

Terraform core reads my `.tf` files, builds a dependency graph, and then talks to AWS through the provider plugin — it's not calling AWS APIs directly itself. The provider binary translates my HCL resource blocks into actual AWS SDK calls. When I run `terraform plan`, core compares my desired state against the state file, and the provider does a refresh call to AWS to check real-world drift before showing me the diff. I always tell juniors: Terraform core is dumb about AWS specifics — all AWS knowledge (API versions, resource schemas) lives in the `aws` provider plugin, which is why provider version pinning matters so much.

Real-time Example:

On one project, we upgraded the AWS provider from 4.x to 5.x without pinning it in a lower environment first, and several `aws_s3_bucket` resources broke because AWS split the resource into separate sub-resources like `aws_s3_bucket_versioning`. Had to rewrite half our S3 module.

Best Practice:

Always pin provider versions using `required_providers` block with a `~>` constraint, and test provider upgrades in dev before touching prod state.

Follow-up Interview Question:

What happens if the provider plugin binary is missing during `terraform apply` in a CI pipeline?

Question 2

Interview Question:

How do you manage multiple AWS providers in a single Terraform configuration, say for multi-region or multi-account setups?

Answer:

I use provider aliasing. In our setup, we had a primary region in `us-east-1` and a DR region in `us-west-2`, so I declared two provider blocks with an `alias` attribute and then referenced the aliased provider explicitly in resources using the `provider` meta-argument. For multi-account, we used `assume-role` in each provider block pointing to different account IDs — this is common in banking clients like Wells Fargo where dev, QA, and prod sit in completely separate AWS accounts for isolation.

Real-time Example:

We had a cross-account VPC peering setup where the requester VPC was in the app account and the acceptor VPC was in the shared services account. I had to alias two providers, each assuming a different IAM role via `sts:AssumeRole`, to create the peering connection and accept it from both sides in the same apply.

Best Practice:

Never hardcode account IDs in provider blocks — pull them from variables or SSM parameters so the same module works across environments without code changes.

Follow-up Interview Question:

How would you structure this if you had 10+ AWS accounts under AWS Organizations?

Question 3

Interview Question:

Explain the difference between a resource and a data source with a real example.

Answer:

A resource is something Terraform creates, owns, and manages the lifecycle of — if I delete it from code and apply, Terraform destroys it in AWS. A data source is read-only; Terraform just fetches existing information and doesn't touch its lifecycle at all. I use data sources constantly to pull things I don't manage in this particular state file — like an existing VPC created by the networking team, or the latest Amazon Linux AMI ID.

Real-time Example:

In one project, the networking team owned the VPC and subnets in their own Terraform state, and my application team just consumed them using `data "aws_vpc"` and `data "aws_subnets"` with tag filters. This avoided state file conflicts between teams — I never had write access to their VPC resources, only read access via data sources.

Best Practice:

Use data sources with tag-based filters instead of hardcoded IDs, so the same code works across dev/QA/prod without manual ID substitution.

Follow-up Interview Question:

What happens if a data source returns zero or multiple matching resources?

Question 4**Interview Question:**

How do you structure variables and outputs for a reusable production module?

Answer:

I split variables into `variables.tf` with proper type, description, and default where applicable, and I mark anything sensitive with `sensitive = true`. For outputs, I only expose what the calling module actually needs — no point outputting every internal attribute. I usually group related variables using object types instead of having 20 flat variables, it keeps the module call clean.

Real-time Example:

For our RDS module, instead of having separate variables like `db_instance_class`, `db_allocated_storage`, `db_engine_version` scattered around, I bundled them into a single object variable called `db_config` with a defined type structure. This made the module call much shorter and self-documenting for other teams consuming it.

Best Practice:

Always add description to every variable and output — during audits at one client, undocumented variables were flagged as a compliance issue since new engineers couldn't understand the module without reading all the code.

Follow-up Interview Question:

How do you handle a variable that should behave differently across environments without duplicating the whole module?

Question 5**Interview Question:**

What's the practical use of `locals` blocks, and how have you used them in production?

Answer:

Locals are for computed values or values I reuse multiple times but don't want to expose as input variables. I use them heavily to build naming conventions and common tags, so every resource in a module gets consistent naming without me repeating string concatenation everywhere.

Real-time Example:

We had a compliance requirement that every AWS resource must have tags like `CostCenter`, `Environment`, `Owner`, and `Project`. Instead of typing these tags on 40+ resources, I created a `local.common_tags` map merged with resource-specific tags using the `merge()` function. When the `CostCenter` tag key changed company-wide, I updated it in one local block instead of 40 places.

Best Practice:

Never put values in locals that should actually be configurable inputs — that's a common mistake juniors make which reduces module reusability.

Follow-up Interview Question:

Can locals reference other locals? Any gotchas with ordering?

Question 6**Interview Question:**

Explain remote state and why you wouldn't use local state in a team environment.

Answer:

Local state means the `.tfstate` file sits on your laptop — the moment a second engineer runs `apply`, you get conflicting state and possibly destroyed resources because nobody else sees your changes. Remote state stores the state file in a shared backend like S3, so the whole team and CI/CD pipeline reads and writes to the same source of truth.

Real-time Example:

Before I joined one project, they were using local state checked into a shared network drive. Two engineers applied changes within minutes of each other, and one's `apply` overwrote the other's state file, effectively "forgetting" about resources that were actually still running in AWS. We spent almost a day reconciling this with `terraform import`. First thing I did was migrate to S3 backend with DynamoDB locking.

Best Practice:

Always use remote state with locking for anything beyond a solo sandbox project — no exceptions, even for POCs that might become real projects.

Follow-up Interview Question:

How do you migrate an existing local state file to a remote S3 backend without losing resources?

Question 7**Interview Question:**

What exactly is stored inside the Terraform state file, and why is it sensitive?

Answer:

The state file is a JSON file that maps every resource in my code to its real-world ID in AWS, plus it stores all the resource attributes Terraform knows about — including things like RDS master passwords, secret ARNs, or private keys if they were ever set as resource attributes. That's why I never store it in plain S3 without encryption, and never commit it to Git.

Real-time Example:

On an audit at a banking client, the security team flagged that our state file in S3 wasn't using SSE-KMS encryption — it was only using default SSE-S3. Since our RDS module output the initial password as a resource attribute, that value was sitting in plaintext-readable state. We enabled KMS encryption on the bucket and rotated the credentials immediately.

Best Practice:

Enable S3 bucket encryption with KMS, block public access, enable versioning for state history, and restrict IAM access to the state bucket to only the CI/CD role and senior engineers.

Follow-up Interview Question:

If your state file gets corrupted or accidentally deleted, how do you recover it?

Question 8**Interview Question:**

How does Terraform state locking work, and describe a real incident you handled related to a stuck lock.

Answer:

State locking prevents two people or pipelines from running apply against the same state simultaneously. With the S3 backend, locking is handled via a DynamoDB table — Terraform writes a lock ID entry before starting an operation and removes it after completion. If the process crashes or the pipeline gets killed mid-apply, that lock entry can get stuck.

Real-time Example:

We had a Jenkins job get killed because the build agent ran out of memory during a large terraform apply. The next pipeline run failed with "Error acquiring the state lock." I had to manually check the DynamoDB table, verify no other apply was actually running by checking with the team, and then run `terraform force-unlock <LOCK_ID>` to clear it. I always confirm with the team on Slack before force-unlocking — doing it while someone else's apply is genuinely running can corrupt state.

Best Practice:

Set reasonable pipeline timeouts and always alert the team before force-unlocking; never make force-unlock a routine step in automation.

Follow-up Interview Question:

What's the risk if you force-unlock while another apply is actually in progress?

Question 9**Interview Question:**

Show me how you'd configure an S3 backend with DynamoDB locking for a production project.

Answer:

I define a backend "s3" block with the bucket name, key path, region, `dynamodb_table` for locking, and `encrypt = true`. I never hardcode this — backend config can't use variables directly, so for multi-environment setups I use partial backend configuration and pass the rest via `-backend-config` files or CLI flags during `terraform init`.

Real-time Example:

```
terraform {  
  backend "s3" {  
    bucket      = "company-terraform-state-prod"  
    key         = "app/prod/terraform.tfstate"  
    region      = "us-east-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

For each environment, I keep separate `backend-dev.hcl`, `backend-qa.hcl`, `backend-prod.hcl` files and run `terraform init -backend-config=backend-prod.hcl` in the pipeline based on the branch or parameter.

Best Practice:

Use a unique key path per environment/component so state files never overlap, and enable versioning on the S3 bucket to roll back a bad state.

Follow-up Interview Question:

Why can't you use variables directly inside the backend block?

Question 10**Interview Question:**

Have you used Terraform Cloud or Terraform Enterprise? How is it different from self-managed S3 backend?

Answer:

Yes, I've used Terraform Cloud on a client project mainly for the remote run execution and built-in state management with a nice UI, policy checks via Sentinel, and VCS-driven runs. Compared to self-managed S3 backend plus Jenkins, Terraform Cloud handles locking, state versioning, and run history out of the box, and it gives structured approval workflows without me building that logic into Jenkins scripts myself. Terraform Enterprise is basically the same thing but self-hosted, which some banking and insurance clients need due to data residency and network isolation requirements.

Real-time Example:

On one engagement, we moved from Jenkins-triggered applies to Terraform Cloud with VCS integration to GitHub. Every PR automatically triggered a speculative plan visible in the PR checks, and merging to main triggered the real apply after a manual approval in the TFC UI. This significantly reduced "it worked on my machine" plan drift issues we used to get with local CLI applies.

Best Practice:

Use Sentinel or OPA policy checks in TFC/TFE to enforce guardrails like "no public S3 buckets" before apply is even allowed to run.

Follow-up Interview Question:

How do Sentinel policies differ from just using `terraform validate`?

Question 11**Interview Question:**

Explain Terraform workspaces and when you would NOT use them for multi-environment management.

Answer:

Workspaces let you maintain multiple state files under the same configuration using `terraform workspace new/select`. They're good for quick, structurally identical environments like short-lived feature branches. But for dev/QA/prod, I generally avoid workspaces because all environments share the same backend config and code path, which increases the blast radius —

a mistake in variable interpolation could apply changes to the wrong workspace if someone forgets to switch.

Real-time Example:

On a project that used workspaces for dev/QA/prod, an engineer ran `terraform apply` thinking he was on the dev workspace but was actually still on prod from a previous session. Since workspace state isn't obviously visible in the terminal, it caused unintended changes to prod security groups. After that incident, we moved to separate directories/backend configs per environment instead of workspaces, which forces the environment to be an explicit part of the pipeline command.

Best Practice:

For anything customer-facing or with compliance requirements, use separate state files and directories per environment rather than workspaces, and make the environment name explicit in your CI/CD pipeline stage.

Follow-up Interview Question:

How do you enforce that an engineer can't accidentally apply to the wrong workspace?

Question 12

Interview Question:

How do you design reusable Terraform modules for an organization with 10+ teams consuming them?

Answer:

I keep modules generic and opinionated only where it truly matters — things like naming standards and tagging are enforced inside the module, but instance sizing, count, and environment-specific values are exposed as variables. I version modules using Git tags and reference them with a pinned version in the `source` argument, so teams don't get surprised by breaking changes.

Real-time Example:

We maintained a central Git repo `terraform-aws-modules` with subfolders for VPC, EKS, RDS, and ALB modules. Each team referenced them like `source = "git::https://github.com/org/terraform-aws-mod"`. When we introduced a breaking change to the VPC module (added mandatory flow logs), we bumped the major version to v3.0.0 and kept v2.x available so existing teams weren't broken until they explicitly opted in.

Best Practice:

Follow semantic versioning strictly for internal modules — patch for bug fixes, minor for backward-compatible additions, major for breaking changes — and document changes in a CHANGELOG.

Follow-up Interview Question:

How would you test a module change before releasing a new version to other teams?

Question 13**Interview Question:**

Give a real example where you used dynamic blocks and why a static block wouldn't have worked.

Answer:

I use dynamic blocks when the number of nested blocks isn't fixed and depends on input variables — a static block only works for a fixed, known structure. Security group rules are the classic case since the number of ingress/egress rules varies per environment.

Real-time Example:

Our security group module took a variable `ingress_rules` as a list of objects (port, protocol, cidr, description), and different applications needed anywhere from 2 to 15 rules. I used a dynamic "ingress" block that iterated over `var.ingress_rules` with `for_each`, so the module worked whether an app needed 3 ports open or 12, without me writing separate module versions.

Best Practice:

Don't overuse dynamic blocks for things that rarely change — it makes the plan output harder to read. Use them specifically where the block count is genuinely variable.

Follow-up Interview Question:

What's the difference between `for_each` inside a dynamic block versus `for_each` on the resource itself?

Question 14**Interview Question:**

When do you use `count` versus `for_each`? What issues have you faced with `count`?

Answer:

I use `count` for simple cases where I just need N identical copies of something, like N subnets across AZs. I use `for_each` when I need to create resources from a map or set of distinct items, especially when items can be added/removed from the middle of the list — because `count` uses index-based addressing, and removing an item from the middle of a list shifts every resource after it, causing Terraform to destroy and recreate resources unnecessarily.

Real-time Example:

We originally used `count` to create IAM users from a list of usernames. When someone removed a username from the middle of the list, Terraform's plan showed it wanted to destroy and recreate several unrelated IAM users because their index positions shifted. That was a scary plan output right before a scheduled apply. We refactored to `for_each` using a set of usernames as the key, and after that, removing one user only affected that specific resource.

Best Practice:

Default to `for_each` with maps or sets for anything where individual items might be added or removed independently; reserve `count` for truly homogeneous, ordinal resources.

Follow-up Interview Question:

How do you convert existing `count`-based resources to `for_each` without destroying and recreating them?

Question 15

Interview Question:

Explain the `lifecycle` block with `create_before_destroy`, `prevent_destroy`, and `ignore_changes`, with production scenarios for each.

Answer:

`create_before_destroy` is for zero-downtime replacements — Terraform creates the new resource first, then destroys the old one, useful for things like launch templates or ASG configurations where you don't want a gap. `prevent_destroy` is a safety guard on critical resources like production RDS or S3 buckets holding customer data — it blocks a `destroy` outright unless you remove the flag. `ignore_changes` tells Terraform to stop reporting drift on specific attributes that get changed outside Terraform, like autoscaling desired capacity managed by an application's scaling policy.

Real-time Example:

Our production RDS instance had `prevent_destroy = true`. An engineer accidentally changed the identifier in code, which would have triggered a destroy-and-recreate. The apply failed immediately with a lifecycle error instead of actually destroying prod data — that flag literally saved us that day. Separately, our ASG had `ignore_changes = [desired_capacity]` because Application Auto Scaling changed that value dynamically, and without ignoring it, every plan showed noisy drift.

Best Practice:

Put `prevent_destroy = true` on every production stateful resource — RDS, S3 with critical data, and DynamoDB tables — as a non-negotiable standard.

Follow-up Interview Question:

How do you actually destroy a resource that has `prevent_destroy` set, when it's genuinely needed?

Question 16

Interview Question:

When is `depends_on` actually necessary if Terraform already builds a dependency graph automatically?

Answer:

Terraform automatically detects dependencies when one resource references another's attribute, like an EC2 instance referencing a security group ID. But sometimes there's a dependency that isn't visible through attribute references — like an IAM policy needing to exist before an application tries to use a role, even though no Terraform attribute directly links them. That's when I use explicit `depends_on`.

Real-time Example:

We had an EKS cluster and a Kubernetes provider resource in the same apply. The Kubernetes provider needed the cluster's OIDC and endpoint to be fully ready, but since it accessed them through a data source instead of a direct resource attribute, Terraform sometimes tried creating Kubernetes resources before the cluster was actually ready to accept API calls. Adding `depends_on = [aws_eks_cluster.main]` on the Kubernetes provider resources fixed the race condition.

Best Practice:

Use `depends_on` sparingly and only when implicit dependency truly doesn't exist — overusing it creates unnecessary serialization and slows down applies.

Follow-up Interview Question:

Can `depends_on` be used at the module level, and what's the caveat?

Question 17

Interview Question:

Walk me through how you'd import an existing AWS resource that was created manually in the console into Terraform state.

Answer:

First I write the resource block in code matching the real resource's expected configuration as closely as I can guess from the console. Then I run `terraform import <resource_address> <resource_id>` to bring it into state. After import, I always run `terraform plan` immediately — it almost never shows zero changes on the first try because my code doesn't perfectly match every attribute, so I iteratively adjust the code until plan shows no diff.

Real-time Example:

A client had an S3 bucket and its bucket policy created manually years ago before they adopted Terraform. I imported the bucket with `terraform import aws_s3_bucket.legacy my-bucket-name`, then ran `plan`, and it showed it wanted to change versioning and lifecycle settings because my code's defaults didn't match reality. I updated the code to match the actual configuration, ran `plan` again until it was clean, then only after that did I feel safe running `apply`.

Best Practice:

Never run `apply` right after `import` without first confirming `plan` shows no unwanted changes — that's the step people skip and it can silently modify a resource that was working fine.

Follow-up Interview Question:

How would you import multiple resources in bulk more efficiently, like using the newer `import` block?

Question 18

Interview Question:

What's the difference between `terraform taint` and `terraform apply -replace`, and which do you use now?

Answer:

`terraform taint` used to mark a resource in state so the next `apply` would destroy and recreate it — but it's deprecated now. I use `terraform apply -replace=<resource_address>` instead, which does the same thing but in a single command without a separate state-modifying step, and it shows up clearly in the plan output before you commit to it.

Real-time Example:

We had an EC2 instance that got into a weird state — SSM agent stopped responding and standard troubleshooting wasn't fixing it, but the instance still showed healthy in EC2 status checks. Rather than manually terminating it and risking Terraform losing track, I ran `terraform apply -replace=aws_instance.app_server`, reviewed the plan to confirm it would only touch that instance, and applied. Clean recreate, no state drift.

Best Practice:

Always review the plan output after `-replace` before confirming `apply` — it's easy to assume only one resource is affected when dependent resources might also get recreated.

Follow-up Interview Question:

What downstream resources might get recreated if the replaced resource has other resources depending on its attributes?

Question 19

Interview Question:

Name the state commands you use most often in real troubleshooting, beyond plan/apply.

Answer:

`terraform state list` to see everything currently tracked, `terraform state show <resource>` to inspect a specific resource's actual stored attributes, `terraform state mv` to rename or move resources without destroying them — especially useful during refactoring — and `terraform state rm` to remove a resource from state without destroying the real infrastructure, which I use when a resource should now be managed elsewhere.

Real-time Example:

We refactored a monolithic root module into smaller modules. Resources that were previously at the root, like `aws_instance.web`, needed to become `module.compute.aws_instance.web` in the new structure. Instead of destroying and recreating (which would've caused downtime), I used `terraform state mv aws_instance.web module.compute.aws_instance.web` for each resource, matching the new module structure, and the plan came back clean with zero resource changes.

Best Practice:

Always back up the state file (or rely on S3 versioning) before running state-modifying commands like `mv` or `rm` — these directly edit state and a mistake can desync it from reality.

Follow-up Interview Question:

What's the risk of using `state rm` if you forget the resource still exists in real infrastructure?

Question 20

Interview Question:

How do you detect and handle configuration drift in a production AWS environment?

Answer:

I run `terraform plan` regularly, even outside of deployment windows, specifically to catch drift — anything changed manually in the console that Terraform doesn't know about shows up as a diff. Some teams schedule a nightly drift-detection Jenkins job that runs `plan` and posts the output to Slack if there's unexpected drift, without actually applying.

Real-time Example:

We caught a case where someone manually added an inbound rule to a security group directly in the AWS console to quickly unblock a vendor integration during an incident, then forgot to update Terraform. Our nightly drift check caught it the next morning — the plan showed Terraform wanted to remove that rule. Instead of blindly applying and breaking the vendor connection, I added the rule properly to code so it matched real-world state and became permanently managed.

Best Practice:

Never let engineers make manual changes in the console for anything Terraform manages, even during incidents — if it's unavoidable during an emergency, follow up immediately by codifying that change.

Follow-up Interview Question:

How would you handle drift on a resource with `ignore_changes` set on the drifted attribute?

Question 21**Interview Question:**

What's actually happening during `terraform init`, and what issues have you debugged around it?

Answer:

`init` downloads provider plugins, initializes the backend, and if modules are referenced from Git or a registry, it downloads those too into `.terraform`. In CI/CD, I've hit issues where `init` fails due to provider version constraints conflicting between the root module and a child module, or backend authentication failures when the CI role doesn't have S3/DynamoDB permissions.

Real-time Example:

In a Jenkins pipeline, `terraform init` started failing intermittently with a "connection reset" error while downloading providers from the public registry — turned out our corporate proxy was rate-limiting the registry. We set up an internal Artifactory mirror for provider plugins and configured `.terraformrc` with a `provider_installation` network mirror block, which fixed reliability significantly.

Best Practice:

For enterprise environments behind proxies or with strict egress rules, use a private provider/module registry mirror instead of relying directly on the public Terraform registry from CI agents.

Follow-up Interview Question:

How do you handle provider version conflicts between a root module and a nested module?

Question 22**Interview Question:**

Describe a scenario where `terraform plan` looked clean but `apply` still failed. How did you debug it?

Answer:

This usually happens due to AWS-side eventual consistency, IAM permission issues that only surface at apply time, or resource limits/quotas. Plan doesn't actually make write calls, so some validation only happens during the real API call in apply.

Real-time Example:

Plan showed a clean diff for creating a new RDS instance, but apply failed with an error about exceeding the account's RDS instance limit — a quota that plan doesn't check against. Another time, apply failed on an IAM role creation with an "AccessDenied" error because our CI role had `iam:CreateRole` but not `iam:PutRolePolicy`, which only surfaced when it tried to attach the inline policy, not during plan's read-only checks.

Best Practice:

Set up CloudWatch service quota alarms for commonly hit limits, and use least-privilege IAM policies for your CI role, but test them thoroughly in dev first, since permission gaps often only show up mid-apply.

Follow-up Interview Question:

How do you handle a partially failed apply where some resources were created and others failed?

Question 23

Interview Question:

How do you manage secrets and sensitive variables in Terraform without exposing them in logs or state?

Answer:

I never hardcode secrets in `.tf` files or `.tfvars` committed to Git. I pull secrets at runtime from AWS Secrets Manager or SSM Parameter Store using data sources, and mark any variable holding sensitive data with `sensitive = true` so it's redacted in plan/apply console output. Even with `sensitive = true`, though, I know the value still lands in the state file in plaintext, so state encryption is equally important.

Real-time Example:

For RDS master passwords, instead of setting a static password variable, we used `manage_master_user_password = true` so AWS itself manages the password in Secrets Manager, and Terraform never even sees the plaintext value. For application secrets like API keys, we stored them in Secrets Manager beforehand and referenced them via data `"aws_secretsmanager_secret_version"` in the app's ECS task definition, so the actual secret value never touched our Terraform code or CI logs.

Best Practice:

Combine `sensitive = true` on variables/outputs with encrypted remote state (SSE-KMS) and tightly scoped IAM access to the state bucket — one control alone isn't enough.

Follow-up Interview Question:

If a sensitive value accidentally got printed in CI logs, what would your incident response look like?

Question 24**Interview Question:**

How do you design an EKS cluster setup in Terraform for production, including node groups and IAM roles?

Answer:

I typically use the community `terraform-aws-modules/eks/aws` module rather than writing EKS from scratch since it handles the OIDC provider setup, IAM roles for service accounts (IRSA), and node group launch templates correctly — hand-rolling EKS has a lot of easy-to-miss IAM trust policy details. I define separate managed node groups for system workloads versus application workloads, using taints/labels so critical add-ons like CoreDNS don't get scheduled on the same nodes as unpredictable app workloads.

Real-time Example:

On one project, we had a separate node group for GPU-based ML workloads with taints so regular pods wouldn't accidentally land there and waste expensive GPU instances. We also set up IRSA so our application pods could call S3 and DynamoDB using scoped IAM roles instead of node-level IAM roles, which was a security requirement from the client's cloud security team since node-wide IAM roles gave every pod on that node the same broad permissions.

Best Practice:

Always use IRSA for pod-level AWS permissions instead of attaching broad IAM policies to the node's IAM role — this follows least privilege at the pod level, not the node level.

Follow-up Interview Question:

How do you handle EKS control plane version upgrades without breaking running workloads?

Question 25**Interview Question:**

How do you structure Terraform code for multi-environment deployments — Dev, QA, UAT, Production?

Answer:

I keep a shared set of modules and separate environment directories, each with its own `.tfvars` file and backend config pointing to a distinct state file. I avoid putting environment-specific logic inside modules — the module stays generic, and environment differences (instance sizes, replica counts, domain names) live purely in the `.tfvars` files per environment.

Real-time Example:

Our repo structure was `modules/` for reusable code, and `environments/dev`, `environments/qa`, `environments/uat`, `environments/prod`, each calling the same modules with different variable values. Prod had `prevent_destroy` guards and multi-AZ RDS, while dev used single-AZ to save cost — but the actual module code was identical, just different `.tfvars`. This meant a module fix tested in dev was guaranteed to behave the same way in prod since the underlying code path never diverged.

Best Practice:

Never let environment directories drift into having different module versions running — pin the same module version across all environments and promote changes through environments in order (`dev` → `QA` → `UAT` → `prod`).

Follow-up Interview Question:

How do you promote a Terraform change from dev to prod safely in your pipeline?

Question 26**Interview Question:**

Describe your Git branching strategy alongside Terraform CI/CD, and how it maps to environments.

Answer:

We used a strategy where `feature/*` branches triggered a `terraform plan` only, visible as a PR check. Merging into `develop` auto-applied to the dev environment. Merging `develop` into `release` triggered QA/UAT deploys with manual approval gates. Merging into `main` triggered production apply, but only after a manual approval step from a lead or release manager — never automatic for prod.

Real-time Example:

On a client project using GitHub Actions, every PR against `main` ran `terraform plan` and posted the plan output as a PR comment using a bot, so reviewers could see exactly what infrastructure would change before approving the code review — not just the code diff. This caught a case where a “small” variable change actually would have recreated an RDS instance, which the reviewer caught from the plan output before merge.

Best Practice:

Never allow direct `terraform apply` to production from a developer’s local machine — production applies should only happen through the pipeline with an approval gate, using a dedicated CI service role, not personal AWS credentials.

Follow-up Interview Question:

How do you handle a hotfix that needs to skip the normal `dev` → `QA` → `prod` promotion flow?

Question 27

Interview Question:

Walk me through a Jenkins pipeline you've built for Terraform, including how you handled approvals.

Answer:

Typical stages: checkout, terraform init, terraform validate, terraform plan (output saved to a file), a manual input step showing the plan for approval, then terraform apply using the saved plan file so what gets applied is exactly what was reviewed — not a fresh plan that could differ if something changed in between.

Real-time Example:

```
stage('Plan') {
    sh 'terraform plan -out=tfplan'
}
stage('Approval') {
    input message: 'Apply this plan?', ok: 'Apply'
}
stage('Apply') {
    sh 'terraform apply tfplan'
}
```

On one incident, we almost approved an apply without noticing the plan showed a security group rule opening port 22 to 0.0.0.0/0 — a teammate accidentally left a debug CIDR value in a .tfvars file. Because the Jenkins plan output was posted visibly before the approval gate, another engineer caught it during review and we fixed the variable before applying.

Best Practice:

Always apply from a saved plan file (-out=tfplan, then apply tfplan), never re-run apply without a plan file, since infrastructure or variables could change between plan and apply otherwise.

Follow-up Interview Question:

How do you prevent the saved plan file from going stale if the apply is delayed by days waiting on approval?

Question 28

Interview Question:

How is your GitHub Actions Terraform pipeline different from Jenkins? What specific GitHub Actions features do you use?

Answer:

Conceptually the stages are similar — init, validate, plan, approval, apply — but GitHub Actions gives me native integration with PR checks and environments with built-in protection rules, so I don't need to build custom approval logic like I would in Jenkins using the input step.

I use GitHub Environments with required reviewers configured directly on the production environment, so the apply job literally cannot run without an approval from a specified reviewer group.

Real-time Example:

We set up a workflow where the `plan` job runs on every PR and comments the plan output using `hashicorp/setup-terraform` action's output, and the `apply` job is a separate job with `environment: production` that only triggers after merge to main, and GitHub blocks it until one of our designated approvers clicks approve directly in the Actions UI. This removed the need for a separate Jenkins approval plugin setup entirely.

Best Practice:

Use OIDC federation between GitHub Actions and AWS IAM instead of storing long-lived AWS access keys as GitHub secrets — removes the risk of leaked static credentials entirely.

Follow-up Interview Question:

How would you set up OIDC-based authentication between GitHub Actions and AWS for a Terraform pipeline?

Question 29

Interview Question:

Describe your rollback strategy when a Terraform apply causes a production incident.

Answer:

Terraform doesn't have a native "rollback" command — my rollback is either reverting the Git commit and re-applying the previous known-good configuration, or restoring state from an earlier S3 version if state itself got corrupted. For truly critical stateful resources like RDS, code-level rollback isn't enough — I rely on automated snapshots/backups as the real safety net, since Terraform reverting code won't undo data loss.

Real-time Example:

An apply modified an ALB listener rule incorrectly, causing traffic to route to the wrong target group and causing a partial outage. I immediately reverted the Git commit, ran plan to confirm it would restore the previous listener rule, and applied — took about 6 minutes total from detection to fix. For RDS-related incidents, we always kept automated daily snapshots plus point-in-time recovery enabled, since that's the only real rollback path for data, not Terraform itself.

Best Practice:

Treat Terraform rollback as "revert code + re-apply," and back it up with actual AWS-native backup/recovery features (RDS snapshots, S3 versioning) for anything involving data — code revert alone doesn't undo data changes.

Follow-up Interview Question:

What if the previous “known good” commit no longer matches real-world state due to drift?

Question 30

Interview Question:

How would you implement Blue-Green or Canary deployment strategy using Terraform for an application on AWS?

Answer:

For Blue-Green with EC2/ASG, I create a completely new ASG and target group (green) alongside the running one (blue), using `create_before_destroy` where relevant, then shift ALB listener rule weights or swap target groups once the green environment passes health checks, and only then terminate the blue ASG. For Canary, I use weighted target groups on the ALB listener, gradually shifting traffic percentage from old to new, which I usually script as a series of applies with different `weight` values rather than one big cutover.

Real-time Example:

For an ECS-based service, we used CodeDeploy’s blue-green deployment type configured through Terraform’s `aws_codedeploy_deployment_group`, which handled the traffic shifting and automatic rollback if CloudWatch alarms triggered during the shift. Terraform’s job was to correctly define the deployment group, the two target groups, and the alarm-based rollback triggers — the actual traffic cutover logic was handled by CodeDeploy, not something I scripted manually in Terraform applies.

Best Practice:

Let a purpose-built service (CodeDeploy, ALB weighted routing, or a service mesh) handle the actual traffic shifting logic, and use Terraform to correctly define the infrastructure and rollback triggers around it — don’t try to hand-roll traffic shifting purely through repeated Terraform applies for real production canary releases.

Follow-up Interview Question:

How would CloudWatch alarms trigger an automatic rollback during a CodeDeploy blue-green shift, and what’s Terraform’s role in that?

Quick Reference: Additional Commands Often Asked About

- `terraform fmt` — I run this as a pre-commit hook and also as a CI check (`terraform fmt -check`) so PRs fail if code isn’t formatted consistently; keeps diffs clean across a team.
- `terraform validate` — Catches syntax and basic configuration errors before plan; I always run this right after init in every pipeline stage, it’s fast and catches dumb mistakes early.
- `terraform refresh` — Now typically done via `terraform plan -refresh-only`, used when I suspect drift and want to see exactly what changed in AWS without proposing any code changes.

- `terraform graph` — I've used this mainly when debugging a confusing dependency ordering issue, piping the DOT output to Graphviz to visualize why resource A is waiting on resource B.
- `terraform console` — Handy for quickly testing an expression or function output, like checking what `cidrsubnet()` returns for a given input, without running a full plan.
- `terraform destroy` — In production, I almost always use `-target` cautiously or scope it to a specific environment's state file; a blind `terraform destroy` at the root without careful scoping is one of the most dangerous commands in the toolkit.

Prepared for interview preparation — covers real production patterns across Terraform, AWS, Jenkins, and GitHub Actions CI/CD workflows.